Section 13

Section 12

Vectors

Lab 12: https://maryash.github.io/135/labs/lab_12.html

Ryan Vaz

Shortcomings of C++ Arrays

Some shortcomings of C++ Static Arrays:

- the *size* of the array needs to be known before declaration
- once initialized, items cannot exceed the size *constraint*
- arrays can't be *returned by value* from a function

In lab 9, we briefly discussed dynamic arrays. Using dynamic memory allocation, a C++ programmer can make their own array-like data structure class that can change in size depending on the number of items in the array. An example of such a dynamic array is a vector.

Introduction to Vector

Vectors are essentially dynamic arrays that can resize automatically at runtime. In addition, vectors can be returned from functions.

To use vectors, include the following: *#include <vector>*

Vector initialization requires the datatype.
For example, here's how to initialize an int vector called vec: *vector<int> vec;*

Unlike arrays, we don't have to declare the size during initialization.

Adding new items to a Vector

```cpp
int main(){
    vector<int> v;          // initialize empty vector
    v.push_back(10);        // add 10 to the back of the vector
    v.push_back(20);        // add 20 to the back of the vector
    v.push_back(30);        // add 30 to the back of the vector
    // v now contains elements [10, 20, 30]
    vector<int> new_v {10, 20, 30}; // list initialize a vector
    // new_v also contains elements [10, 20, 30]
}
```

Accessing elements in a vector

By value: Use [ ] similar to arrays. For example: vec[0] would return item in 0 index of the vector vec as the datatype of the vector.

By reference: Use *at()* function to get an element by reference. For example: vec.at(0) to get item in 0 index.

First item: *front()* function returns the first element by reference

Last item: *back()* function returns the last element by reference

Accessing elements in a vector

push_back(n) : adds element n at the back (end) of the vector
pop_back() : removes the last element in the vector
clear() : removes everything from the vector

There are other functions within the vector class. You can learn more about those in the official documentation of vector.

As stated before, vectors are dynamic arrays. You can implement your own vector class using dynamic arrays.

Pairwise Sum

Implement the following function:
vector<int> sumPairWise(const vector<int> &v1, const vector<int> &v2);

Returns a vector of integers whose elements are the pairwise sum of the elements from the two vectors passed as arguments. If a vector has a smaller size than the other, consider extra entries from the shorter vectors as 0.

```
int main(){
    vector<int> v1{1,2,3};  // initialize v1 as [1, 2, 3]
    vector<int> v2{4,5};    // initialize v2 as [4, 5]
    sumPairWise(v1, v2);    // returns [5, 7, 3]
}
```

Pairwise Sum: Pseudocode

```
vector<int> sumPairWise(const vector<int> &v1, const vector<int> &v2) {
    // create two empty vectors small and large
    if (/* size of v1 is less than size of v2 */){
        // set small equal to v1 and large equal to v2
    }
    else {
        // set small equal to v2 and large equal to v1
    }
    for (/* i starts at 0 and goes upto the size of small */) {
        large[i] = large[i] + small[i];
    }
    return large;
}
```

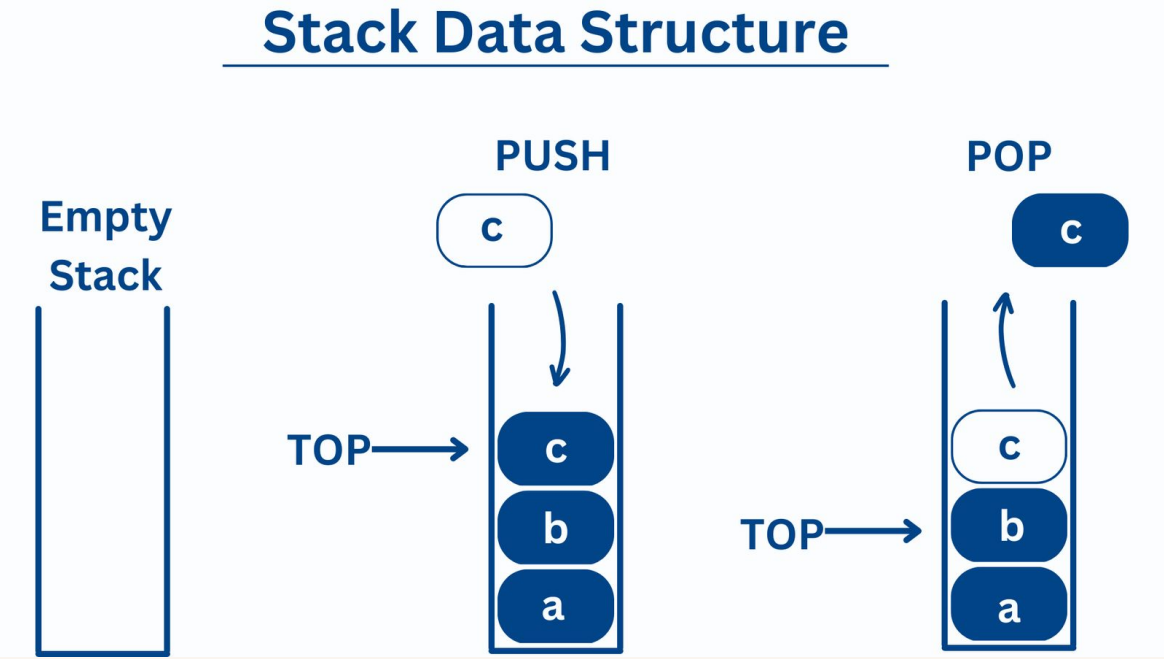
Other data types: Stack

A stack is essentially a list of values, but with a more constrained set of operations available on that list. Most notably, we can only access and manipulate one element in the data structure: the last element that was inserted into the stack (which, in stacks, is known as the top of the stack).

Some examples of stacks used in the real world are:

- Undo algorithms in a text editor
- Memory management in a computer
- Function call evaluation, which originates from the function call stack

Other data types: Stack

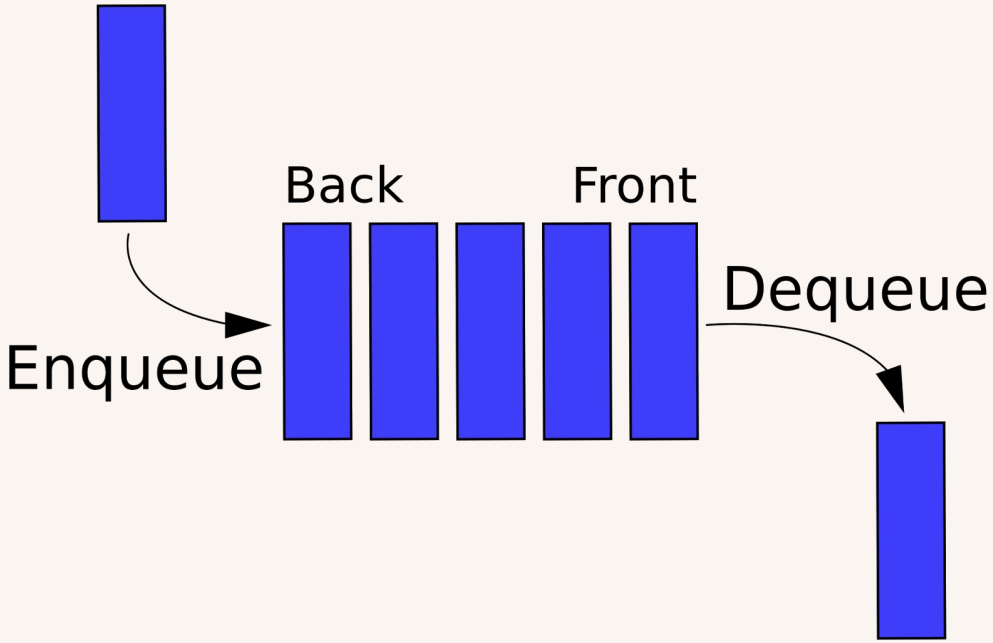


Other data types: Queue

A queue is like the typical queue you encounter when waiting in line for something: elements can only enter the queue through the back of the queue and can only exit the queue through the front. More specifically, the only allowed operations with a queue are:

- Creating an empty queue
- Enqueueing a value at the back of the queue
- Dequeueing a value at the front of the queue
- Peeking at the value at the front of the queue
- Checking the size of queue, including whether it is empty

Other data types: Queue



Stacks/Queues vs Arrays/Vectors

Stacks / Queues should be used when you need to manage the data in a specific manner, for the sake of CS135 you won't need to worry about these. Going forward, when there's an opportunity to use a Stack or Queue, it should be seized due to its performance improvement over standard arrays.

Stacks and Queues are very powerful data types that will be useful for writing more efficient and fast programs. They are commonly used as interview questions.

https://leetcode.com/problem-list/stack/
https://leetcode.com/problem-list/queue/
https://leetcode.com/problem-list/array/